TEFR-YOLO: Texture Encoding and Feature Reassembly for Fabric Defect Detection

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, and Chunhua He

***Abstract*—Circular knitting machines are essential to textile production, but needle or yarn abnormalities can generate defects continuously, allowing fabric waste to accumulate until the problem is resolved. To address this problem, we develop a fabric defect detection system comprising a Hall sensor, an RK3566 edge device, an industrial camera, and LED illumination. The Hall sensor detects machine rotation and signals the edge device to activate the camera and LED illumination. If a defect is detected, the edge device sends a stop signal to the knitting machine's control system through a wired connection. It also uploads the defect image to an Internet of Things platform for defect statistics and production-quality analysis. To provide a compact detector for this system, we propose TEFR-YOLO, which combines directional texture encoding (DTE) with local texture-guided feature reassembly (LTFR). DTE models horizontal, vertical, and compact local texture disruptions, whereas LTFR replaces fixed top-down upsampling with content-adaptive reconstruction to better retain narrow defect boundaries and weak texture disruptions during multi-scale fusion. On a dataset collected from a real textile production line, TEFR-YOLO achieves 66.3% AP and 96.9% AP50, exceeding YOLOv8n by 7.0 and 2.8 percentage points, respectively, with 3.39 million parameters. The deployed model runs at 28 frames per second on the RK3566 edge device. These results support timely machine intervention and cloud-based quality analysis during fabric production.**

***Index Terms*—Industrial defect detection, YOLO, deep learning, edge computing, Internet of Things.**

I. Introduction

CIRCULAR knitting machines are widely used in weft-knit fabric production and support high-rate, large-scale manufacturing [1], [2]. The resulting fabrics are used in everyday garments such as T-shirts and sportswear [2], [3]. Conventional quality checks are often performed only after knitting is complete; by then, persistent needle or yarn abnormalities may already have caused substantial fabric loss. Detecting defects during knitting is therefore essential to prevent further waste.

Single-stage YOLO detectors jointly predict defect categories and bounding boxes, making them attractive for industrial defect detection under latency constraints [4]. Subsequent developments, including anchor-free prediction in YOLOX [5] and training-oriented architectural improvements in YOLOv7 [6], have broadened the accuracy–efficiency choices available to deployment systems. YOLO-based methods have been applied to small steel-surface defects [7], steel-surface defect detection [8], and cracking defects in large stamped parts [9]. These applications demonstrate the relevance of explicit bounding-box localization to industrial defect detection, while also showing that detector design must account for the appearance and deployment constraints of the target process.

For fabric defects, existing adaptations can be organized around lightweight design, feature extraction, and multi-scale detection. AYOLOv7-tiny and YOLOV4-TinyS emphasize compact architectures for efficient fabric defect detection [10], [11]. Deformable feature extraction accommodates irregular defect geometry [12], [13], and DCFE-YOLO uses dynamic snake convolution to strengthen elongated and subtle defect responses [14]. Additional detection scales and enhanced fusion address small defects and cross-scale information loss [15]–[18]; attention and receptive-field refinements provide further representation improvements [19], [20]. Rather than assuming that existing detectors cannot represent directional defects, our focus is on a compact, explicit encoding of orthogonal texture disruptions and its integration with adaptive top-down reconstruction for circular-knitting defect detection.

Two design considerations motivate this approach. First, under the installed camera geometry, circular-knitted fabrics exhibit repetitive patterns with dominant horizontal and vertical image orientations. Defects disrupt this regular texture in horizontally extended, vertically extended, or compact local forms such as holes. Standard convolutional feature extraction can learn these patterns implicitly, but does not explicitly separate directional and compact texture evidence. Second, in a conventional YOLO pyramid using nearest-neighbor upsampling, higher-resolution features are reconstructed by a content-independent copying rule. This rule cannot adapt its aggregation weights to the local texture and may inadequately retain narrow boundaries or weak disruptions. These considerations motivate explicit directional texture encoding during feature extraction and content-adaptive reconstruction during top-down fusion.

We develop TEFR-YOLO to introduce texture-specific inductive biases at both feature encoding and feature reconstruction stages. A directional texture encoding (DTE) module follows selected backbone stages, modeling horizontal, vertical, and compact local evidence through complementary branches. Residual fusion retains a direct path for the baseline representation while augmenting it with texture cues. A local texture-guided feature reassembly (LTFR) pyramid then replaces the two fixed top-down upsampling operations with location-specific neighborhood aggregation. Together, these modifications preserve the lightweight single-stage YOLO detection paradigm while adapting its encoding and reconstruction stages to knitted-texture disruptions.

Beyond detector accuracy, detection during production

Kangzhao Tan, Jiarui Zhang, Junjie Yu, and Chunhua He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China (e-mail: tankangzhao@mails.gdut.edu.cn; zhangjiarui1@mail2.gdut.edu.cn; yujunjie2@mails.gdut.edu.cn; hechunhua@pku.edu.cn).

Heng Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China (e-mail: heng.wu@foxmail.com).

Corresponding author: Chunhua He.

requires sensing to follow machine operation and defect decisions to reach the machine controller without a cloud round trip. Consistent with the local processing rationale of edge computing [21], we deploy TEFR-YOLO on an RK3566 edge device. A detected defect triggers automatic wired stop control before the result is displayed for on-site operator review and troubleshooting. Defect records and device states are subsequently available to an IoT cloud platform for supervision, statistics, and production-quality analysis. The cloud supports monitoring and analysis; it is not a prerequisite for a local stop decision.

The main contributions of our work are as follows.

- We introduce a directional texture encoding (DTE) module that explicitly models horizontal, vertical, and compact local texture cues and fuses them with the baseline backbone representation, providing a lightweight texture-specific inductive bias for circular-knitted defect detection.
- We develop a local texture-guided feature reassembly (LTFR) pyramid that replaces fixed nearest-neighbor upsampling with location-adaptive reconstruction, enabling content-adaptive reconstruction of local defect structures during top-down multi-scale fusion.
- We design and implement a production-line defect detection system integrating Hall sensing, LED illumination, industrial imaging, RK3566 edge inference, wired machine control, and an IoT cloud platform. Production-line data and on-device measurements evaluate the detector and its deployment feasibility.

II. Proposed Method

A. TEFR-YOLO Architecture

TEFR-YOLO is instantiated on YOLOv8n [22]. Baseline selection considers hardware compatibility as well as model complexity: Rockchip's official INT8 benchmark reports 34.0 FPS for YOLOv8n and 20.6 FPS for YOLO11n on RK3566/RK3568 at 640×640 input [23]. YOLOv5n is faster in that benchmark (39.7 FPS), so YOLOv8n is not selected on a claim of maximum throughput. Its supported deployment path, compact architecture, and modular backbone and pyramid instead provide a practical basis for the proposed modifications. These vendor measurements concern model inference and are distinct from our deployed-model measurement.

Fig. 1 shows the overall architecture. DTE follows the output of each of the four backbone C2f stages at strides 4, 8, 16, and 32. The internal C2f bottlenecks and their feature-aggregation paths remain intact; directional processing is not repeated inside every bottleneck. The spatial pyramid pooling stage is also retained. The neck follows the top-down and bottom-up multi-scale aggregation pattern associated with feature pyramids [24], [25]. LTFR replaces only the two top-down nearest-neighbor operations: one reconstructs stride-32 features at stride 16, and the other reconstructs the fused stride-16 features at stride 8. Each performs a single $2\times$ upsampling operation. Lateral concatenation, neck C2f blocks, bottom-up aggregation, and the three-scale detection head are unchanged. For a 640×640 input, the final detection features have spatial sizes 80×80, 40×40, and 20×20.

B. Directional Texture Encoding

The DTE module in Fig. 2 encodes a defect as a departure from the surrounding knitted texture. Horizontal and vertical branches provide orientation-specific support, while a compact branch responds to local disruptions without favoring an elongated shape. Depthwise filtering separates spatial processing from subsequent channel mixing, following the efficiency principle of depthwise-separable networks [26].

For a C2f output $X \in \mathbb{R}^{C\times H\times W}$, the three branch responses are

$$F_h = \mathrm{DW}_{1\times 7}(X), \tag{1}$$

$$F_v = \mathrm{DW}_{7\times 1}(X), \tag{2}$$

$$F_l = \mathrm{DW}_{3\times 3}(X). \tag{3}$$

Here, DW denotes depthwise convolution followed by batch normalization and SiLU activation. Unit stride and symmetric padding preserve the spatial dimensions. The 1×7 and 7×1 kernels extend support along the horizontal and vertical image axes, respectively, while the 3×3 branch encodes compact neighborhoods. All branches produce C channels. Their concatenation is fused as

$$F_d = \phi\left([F_h, F_v, F_l]\right), \tag{4}$$

where $[\cdot]$ denotes channel concatenation and ϕ is a 1×1 convolution mapping $3C$ channels to C, followed by batch normalization and SiLU. This learnable fusion mixes the three types of texture evidence rather than assigning a fixed branch to a particular defect class. A residual connection [27] gives

$$Y = X + F_d. \tag{5}$$

The identity path allows the incoming C2f representation to pass directly to the output, while the residual branch learns a texture-dependent correction. DTE is related to the use of asymmetric spatial kernels in ACNet [28]; here, orthogonal depthwise filtering, compact local filtering, concatenation, and residual fusion are assembled after the C2f output. The horizontal and vertical terms refer to image orientations, not to warp and weft yarn systems. The kernel length of seven defines the configuration evaluated in this study; its optimality is not assumed.

C. Local Texture-Guided Feature Reassembly

Top-down fusion transfers semantic information from coarse maps to finer maps containing more spatial detail. With nearest-neighbor upsampling, this transfer repeats source features using fixed assignments, irrespective of whether a neighborhood contains regular texture or a defect boundary. LTFR instead predicts a normalized local reconstruction kernel for each output location, as shown in Fig. 3.

The reassembly formulation follows CARAFE [29]. We use this mechanism at both top-down fusion stages with a compression ratio of four, scale factor $s = 2$, and reassembly size $k = 5$. Given $X \in \mathbb{R}^{C\times H\times W}$, the kernel-prediction branch first compresses the channels to $C_r = \max(\lfloor C/4 \rfloor, 16)$:

$$X_c = E_{1\times 1}(X). \tag{6}$$

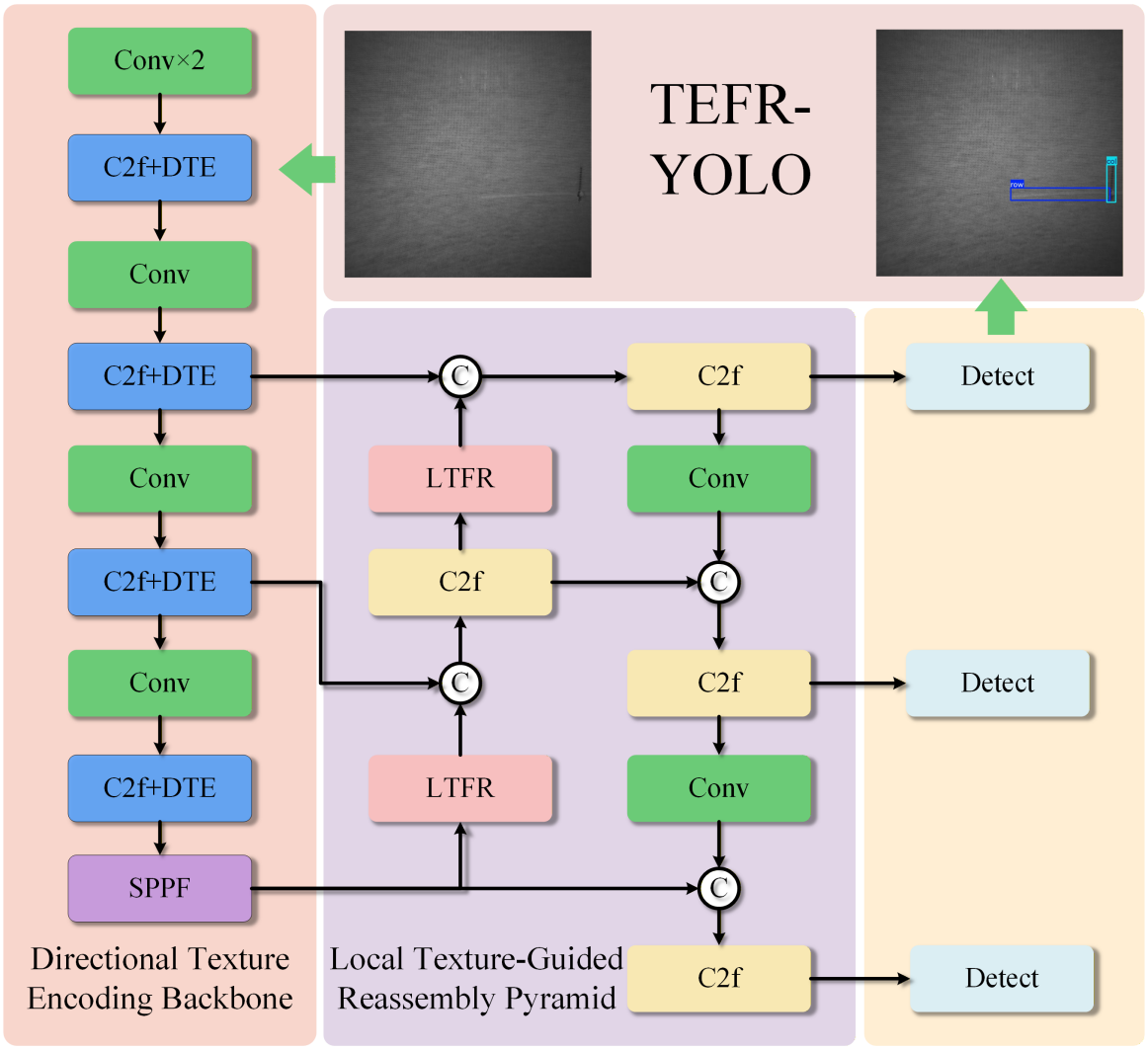

Fig. 1. Architecture of TEFR-YOLO. DTE follows the backbone C2f stages, and LTFR replaces the two top-down 2× upsampling operations. The neck fusion blocks, bottom-up path, and three-scale detection head are retained from YOLOv8n.

The compressor $E_{1\times1}$ includes batch normalization and SiLU. A linear 3×3 convolution then predicts $s^2k^2 = 100$ kernel logits at each input location:

$$K' = E_{3\times3}(X_c). \tag{7}$$

Pixel shuffle rearranges these logits onto the enlarged grid,

$$\widehat{K} = \mathrm{PS}(K'), \tag{8}$$

where $\widehat{K} \in \mathbb{R}^{k^2\times sH\times sW}$. Softmax along the k^2 neighborhood dimension produces location-specific weights:

$$K_{i,j} = \mathrm{softmax}(\widehat{K}_{i,j}). \tag{9}$$

The normalization constraint is

$$\sum_{u=1}^{k^2} K_{i,j,u} = 1, \qquad K_{i,j,u} \geq 0. \tag{10}$$

In parallel, $\mathrm{Unfold}_{k\times k}$ extracts source neighborhoods with unit stride and zero padding of $(k-1)/2$. For a zero-based output location (i, j), the corresponding source neighborhood is

$$P_{i,j} = \mathrm{Unfold}_{k\times k}(X)_{\lfloor i/s\rfloor,\lfloor j/s\rfloor}, \tag{11}$$

where $P_{i,j} \in \mathbb{R}^{C\times k^2}$. This alignment replicates source neighborhoods onto the enlarged grid without replacing the learned output-specific weights. The same spatial weights are applied to every feature channel:

$$Y_{c,i,j} = \sum_{u=1}^{k^2} K_{i,j,u} P_{c,i,j,u}. \tag{12}$$

The output has shape $C \times sH \times sW$. Equation (12) allows reconstruction weights to vary with local texture content, so defect boundaries and regular knitted regions can be aggregated differently. Channel compression limits kernel-prediction overhead, while neighborhood reassembly introduces adaptive spatial mixing before lateral fusion. In TEFR-YOLO, the role of this CARAFE-based mechanism is specifically to provide content-adaptive reconstruction at the two top-down pyramid stages.

DTE and LTFR play complementary roles: DTE enhances direction-sensitive texture representation in the backbone, while LTFR performs content-adaptive reconstruction of these features across pyramid scales.

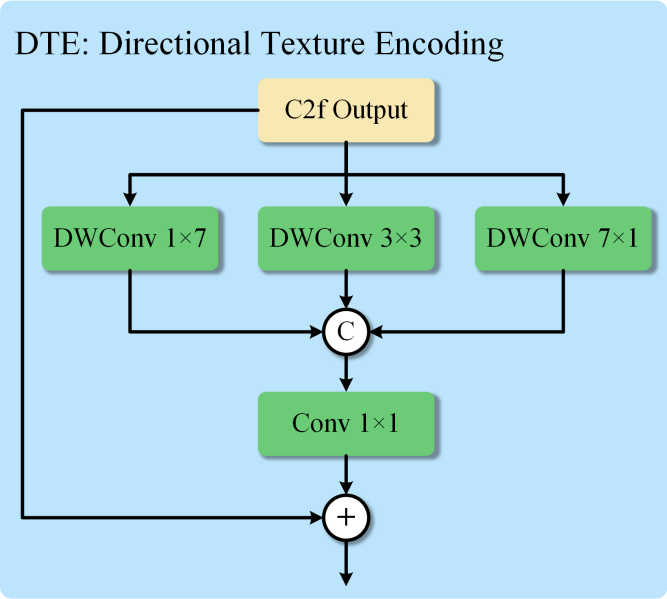

Fig. 2. DTE structure. A C2f feature is processed by horizontal, vertical, and local depthwise branches, followed by channel concatenation, 1×1 fusion, and residual addition.

D. Closed-Loop Detection System

Fig. 4 shows the installed system. The operator presses the start button and the circular knitting machine begins rotating. The Hall sensor detects rotation and sends a signal to the RK3566 edge device, which switches on the LED light, acquires the camera video stream, and enters detection mode. Once a defect is detected, the edge device issues a stop command through its wired connection to the machine controller to interrupt further defect propagation. The defect image and detection result are displayed locally for operator review and troubleshooting. The operator resolves the problem and restarts the machine, completing the operational cycle. Human review follows automatic stop control rather than authorizing it in advance.

Deployment uses the conversion path PyTorch $\rightarrow$ ONNX $\rightarrow$ INT8 RKNN. The trained model is exported with a fixed 640×640 input and converted using Rockchip's RKNN-Toolkit2 [30]. Post-training INT8 quantization provides a reduced-precision representation for NPU execution; integer inference is a widely used approach to efficient neural-network deployment [31]. The RKNN model is executed by the Android application on the RK3566 edge device. Deployment throughput is reported separately from the floating-point checkpoint accuracy evaluated in Section IV.

The cloud platform receives defect records and device states for centralized supervision, defect statistics, and production-quality analysis. Fig. 5 illustrates device-state monitoring, remote configuration, and review of uploaded defect images. These functions provide an operational digital representation of the installed equipment; no automatic process-parameter optimization is assumed. Detection and wired stop control remain local and do not require cloud availability.

III. Experimental Setup

A. Dataset and Implementation

Images were collected during operation of a real circular-knitting production line using the arrangement in Section II-D. A global-shutter industrial camera captures 1280×1024 frames at 30 FPS, approximately 10 cm from the fabric, with a field of view of approximately 16×14 cm. The project team retained defect-containing images and manually annotated axis-aligned bounding boxes. The engineering labels *row*, *col*, and *hole* denote predominantly horizontal disruptions, vertical disruptions, and holes, respectively. These short labels are retained in detection displays, while their visual meanings are used in the analysis.

The dataset contains 403 images and 468 annotated instances. Its nonoverlapping training, validation, and test subsets contain 282, 40, and 81 images, respectively (Table I). Validation data are used for checkpoint selection, and detection accuracy is reported on the held-out test set of 81 images and 91 instances. Table II summarizes the implementation settings. TEFR-YOLO and its direct YOLOv8n baseline load compatible weights from `yolov8n.pt`; newly introduced layers use framework initialization. Other models use the available official pretrained initialization in their respective repositories. Training recipes are not assumed to be identical across repositories.

TABLE I
Image-level split and defect-instance counts in the real production-line dataset.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

B. Evaluation Metrics

We report average precision across the ten IoU thresholds used in COCO evaluation [32], together with AP at IoU 0.50. For a predicted box B_p and a ground-truth box B_g, intersection over union is $\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g| / |B_p \cup B_g|$. At threshold t, a correct detection must have the correct class and match an otherwise unmatched ground-truth instance. Let $\mathrm{AP}_c(t)$ denote the area under the interpolated precision–recall curve for class c, expressed on $[0, 1]$. For $N_c = 3$ classes, the percentage-valued metrics are

$$\mathrm{AP} = \frac{100}{10 N_c} \sum_{c=1}^{N_c} \sum_{t \in \mathcal{T}} \mathrm{AP}_c(t),$$

$$\mathrm{AP50} = \frac{100}{N_c} \sum_{c=1}^{N_c} \mathrm{AP}_c(0.50), \tag{13}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$. AP therefore evaluates localization over increasingly strict overlap requirements, whereas AP50 uses a single, less restrictive threshold. Differences between percentage-valued AP scores are reported in percentage points.

The comparisons include lightweight YOLO models [22], [33], DETR [34], Deformable DETR [35], Faster R-CNN [36],

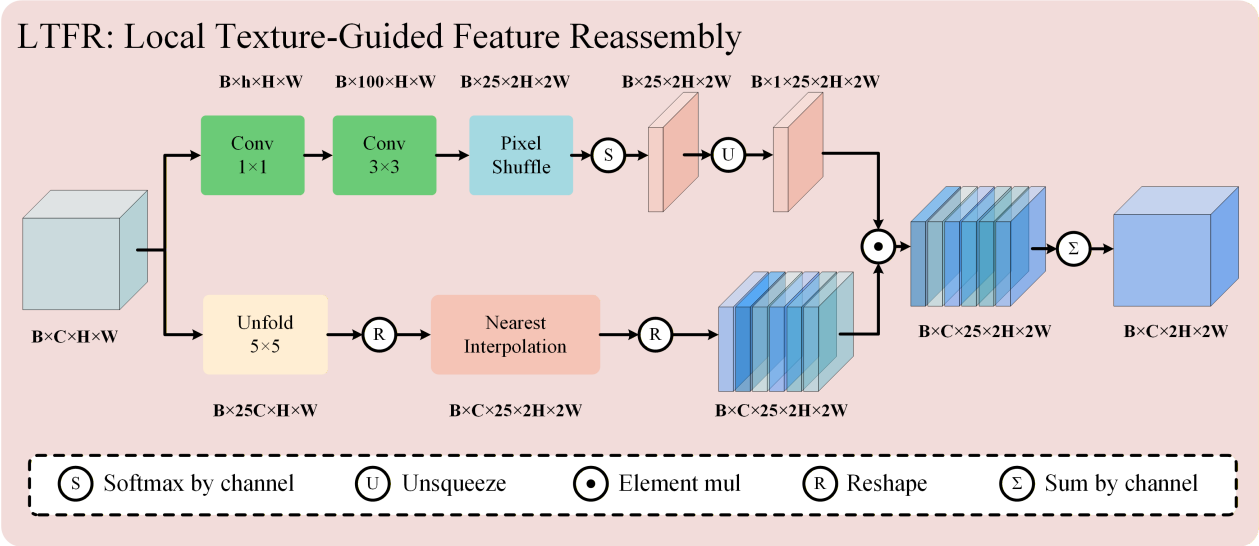

Fig. 3. LTFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds and aligns local feature patches. Their element-wise product is summed along the kernel dimension to produce a $2\times$ upsampled feature.

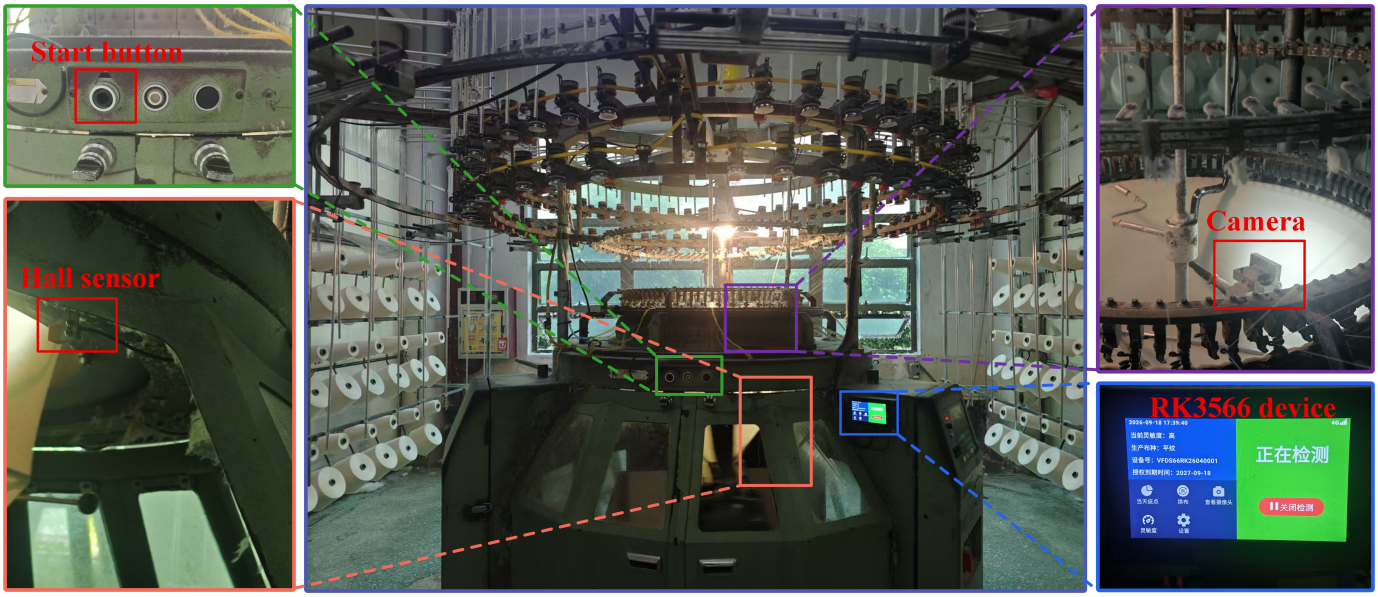

Fig. 4. Actual deployment on the circular knitting machine. The insets highlight the start button, Hall sensor, industrial camera, and RK3566 edge device, respectively.

and an R18 model from the RT-DETRv2 implementation [37], denoted RT-DETR-R18. YOLOv5n refers to the implementation evaluated within Ultralytics, not a separate run of the original YOLOv5 repository.

All models are tested on the same held-out images using their native evaluators. Ultralytics runs retain at most 16 detections per image; RT-DETR uses a COCO maximum of 100. The retained MMDetection reports use maxima of 100 for AP and 1000 for AP50. Native interpolation and postprocessing also differ. Consequently, cross-framework results provide contextual comparisons rather than a fully harmonized benchmark; the direct YOLOv8n comparison and module ablations share the same evaluation implementation.

Parameter counts and approximate FLOPs at input shape $[1, 3, 640, 640]$ characterize model scale. YOLO counts come from fused model summaries, whereas the other counts follow their profiling reports. FLOPs use framework-native estimates, with approximately two operations per multiply–accumulate; unsupported operators may be omitted. These figures do not establish a cross-framework latency ranking. On-device throughput is measured separately for the deployed TEFR-YOLO.

IV. Results and Discussion

A. Comparison With Other Detectors

Table III presents the test results. TEFR-YOLO achieves 66.3% AP and 96.9% AP50, compared with 59.3% and 94.1% for YOLOv8n. The improvements of 7.0 and 2.8 percentage

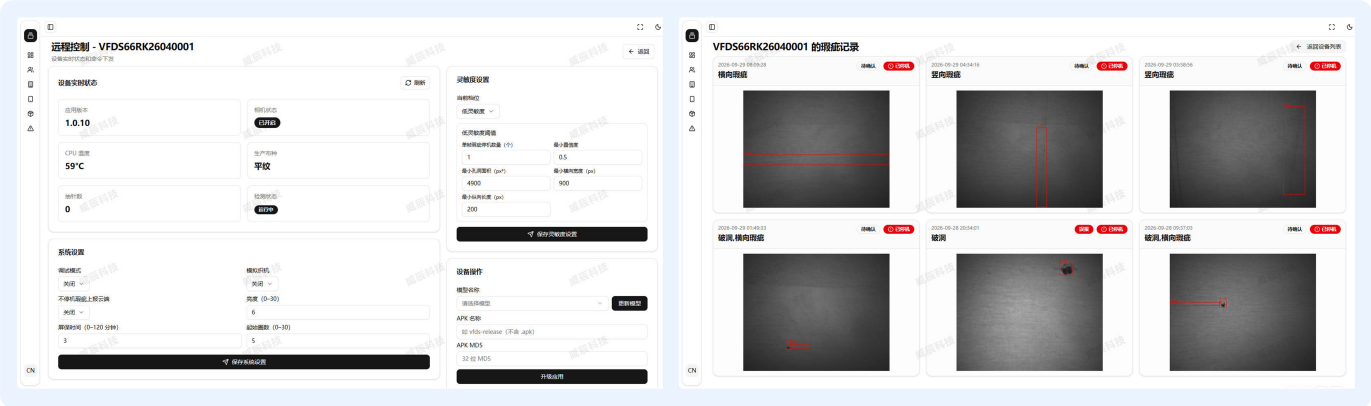

Fig. 5. IoT cloud interface: device-state monitoring and remote configuration (left), and uploaded defect records (right). Cloud functions support centralized supervision and quality analysis; local detection and wired stop control do not depend on cloud communication.

TABLE II
IMPLEMENTATION PROTOCOL. TRAINING SETTINGS DESCRIBE THE ULTRALYTICS EXPERIMENTS; DEPLOYMENT SETTINGS DESCRIBE THE INSTALLED EDGE DEVICE.

Category	Parameter	Specification
1. Training setup	Hardware	Intel i7-13700KF; NVIDIA RTX 3090 (24 GB)
	Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
2. Deployment setup	Hardware	Rockchip RK3566
	System	Android 13
	Quantization	INT8
3. Data protocol	Input resolution	Fixed at 640×640
	Augmentation	Standard Ultralytics geometric and photometric augmentation; no augmentation hyperparameter was manually overridden
4. Training dynamics	Batch	16
	Maximum epochs	300
	Early-stopping patience	80 epochs
	Optimizer	Ultralytics automatic optimizer selection

points require 0.38 M additional parameters and approximately 0.8 G additional FLOPs. The larger gain under the stricter AP metric suggests that the modification improves localization quality beyond increasing the number of detections accepted at IoU 0.50.

Among the evaluated compact YOLO variants, TEFR-YOLO has the highest reported AP. RT-DETR-R18 attains a higher AP of 67.7%, with 20.09 M parameters and 60.72 G estimated FLOPs, compared with 3.39 M and 8.90 G for TEFR-YOLO. Thus, the proposed design occupies a lower-complexity operating point rather than providing the highest accuracy among all architectures. Cross-framework score differences should be read together with the evaluator qualifications in Section III-B.

Fig. 6 visualizes the relationship between accuracy and model complexity. The logarithmic horizontal axes make the compact YOLO models and larger comparison models visible within the same panels. The plot summarizes the tabulated estimates; it does not replace hardware measurements or remove the differences between evaluation implementations.

B. Component Ablation

Table IV separates the effects of texture encoding and feature reassembly under the common YOLOv8n recipe. DTE raises AP from 59.3% to 64.1%, an increase of 4.8 percentage points, with approximately 0.27 M additional parameters. Its AP50 is close to that of the baseline, suggesting that its principal benefit in this experiment concerns stricter localization. LTFR alone achieves 60.7% AP and 94.9% AP50, improving the baseline by 1.4 and 0.8 percentage points, respectively, with approximately 0.10 M additional parameters.

The complete model reaches 66.3% AP, exceeding DTE alone by 2.2 points and LTFR alone by 5.6 points. This pattern is consistent with complementary roles for backbone encoding and pyramid reconstruction. The evidence concerns the two complete modules: it does not isolate the contribution of each DTE branch, establish an optimal directional kernel length, or compare LTFR against every alternative upsampler.

C. Per-Class and Qualitative Results

The class-wise AP values of TEFR-YOLO are 61.7%, 73.8%, and 63.4% for *row*, *col*, and *hole*, respectively. The corresponding YOLOv8n values are 62.2%, 59.6%, and 56.2%. The aggregate improvement therefore comes primarily from vertical defects (+14.2 points) and holes (+7.2 points), while horizontal-defect AP decreases slightly by 0.5 points. This result supports a more specific interpretation than uniform improvement across all morphologies. The vertical category contains only 12 test instances, so the magnitude of its gain should not be generalized beyond the evaluated sample without further data.

Fig. 7 compares seven selected test images using common colors, line widths, and a display confidence threshold of 0.25. These examples allow visual comparison of missed detections, extra predictions, and box alignment against repetitive texture.

TABLE III
DETECTION RESULTS ON THE COMMON TEST SET. BOLD AND UNDERLINED VALUES INDICATE THE HIGHEST AND SECOND-HIGHEST REPORTED SCORES, RESPECTIVELY. CROSS-FRAMEWORK EVALUATORS DIFFER AS DESCRIBED IN SECTION III-B.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
Lightweight YOLO Detectors				
YOLOv5n†	2.50	7.10	93.1	58.4
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
Transformer Detectors				
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	<u>96.8</u>	64.3
Two-Stage Detector				
Faster R-CNN-R50	41.13	181.45	96.1	64.7
Baseline				
YOLOv8n	3.01	8.10	94.1	59.3
Ours				
TEFR-YOLO	3.39	8.90	**96.9**	<u>66.3</u>

†Ultralytics implementation. RT-DETR-R18 uses the RT-DETRv2 codebase. Parameter counts and FLOPs follow the profiling conventions stated in Section III-B.

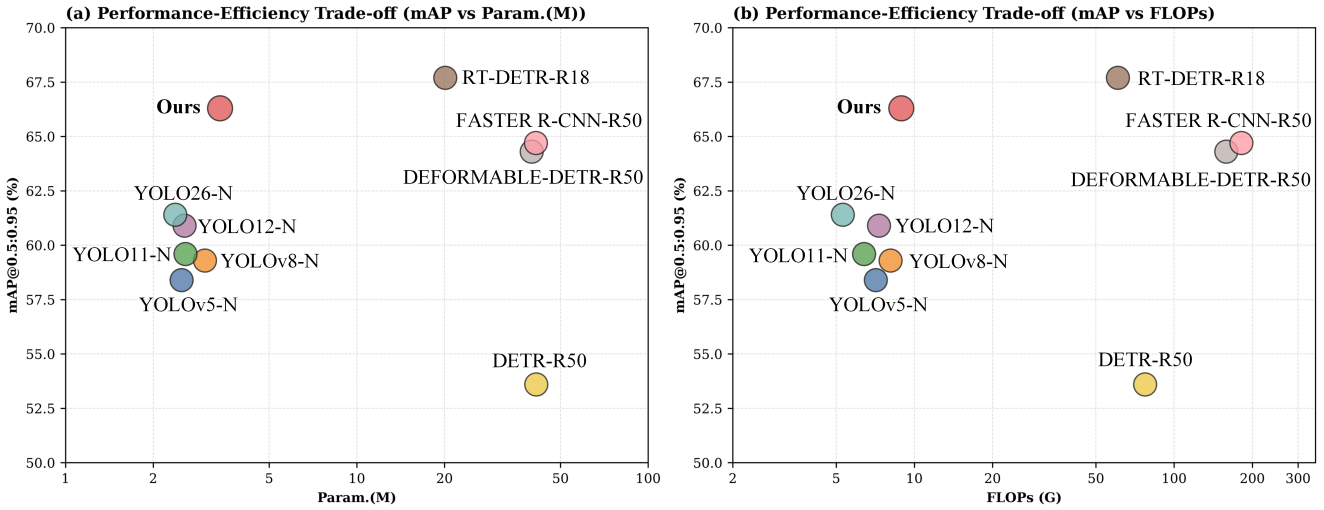

Fig. 6. Accuracy versus parameter count (left) and approximate FLOPs (right). Both horizontal axes are logarithmic. AP corresponds to the mAP@0.5:0.95 notation in the plot; Ours denotes TEFR-YOLO.

TABLE IV
COMPONENT ABLATION. PARAMETERS ARE IN MILLIONS, FLOPS IN BILLIONS, AND AP50/AP IN PERCENT.

Variant	DTE	LTFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ DTE	✓	–	3.28	8.8	94.2	64.1
+ LTFR	–	✓	3.11	8.2	94.9	60.7
TEFR-YOLO	✓	✓	3.39	8.9	**96.9**	**66.3**

They illustrate individual predictions rather than establish a population-level ranking; the quantitative comparison uses the full test set.

Fig. 8 provides complementary, illustrative activation maps generated by the local LayerCAM-based implementation [38]. Positive class-score gradients weight the feature maps entering the three detection scales. Maps are spatially normalized for each candidate and scale, combined by a pixelwise maximum, and overlaid on the original image. The same procedure is used for both models, considering up to 16 highest-scoring candidates above 0.25, with a highest-score fallback if none exceed the threshold. The selected rows originate from the validation, training, and test subsets, respectively, and are not treated as additional held-out evidence. Per-map normalization and candidate selection also mean that heat intensity is not directly comparable as an absolute measure of model quality.

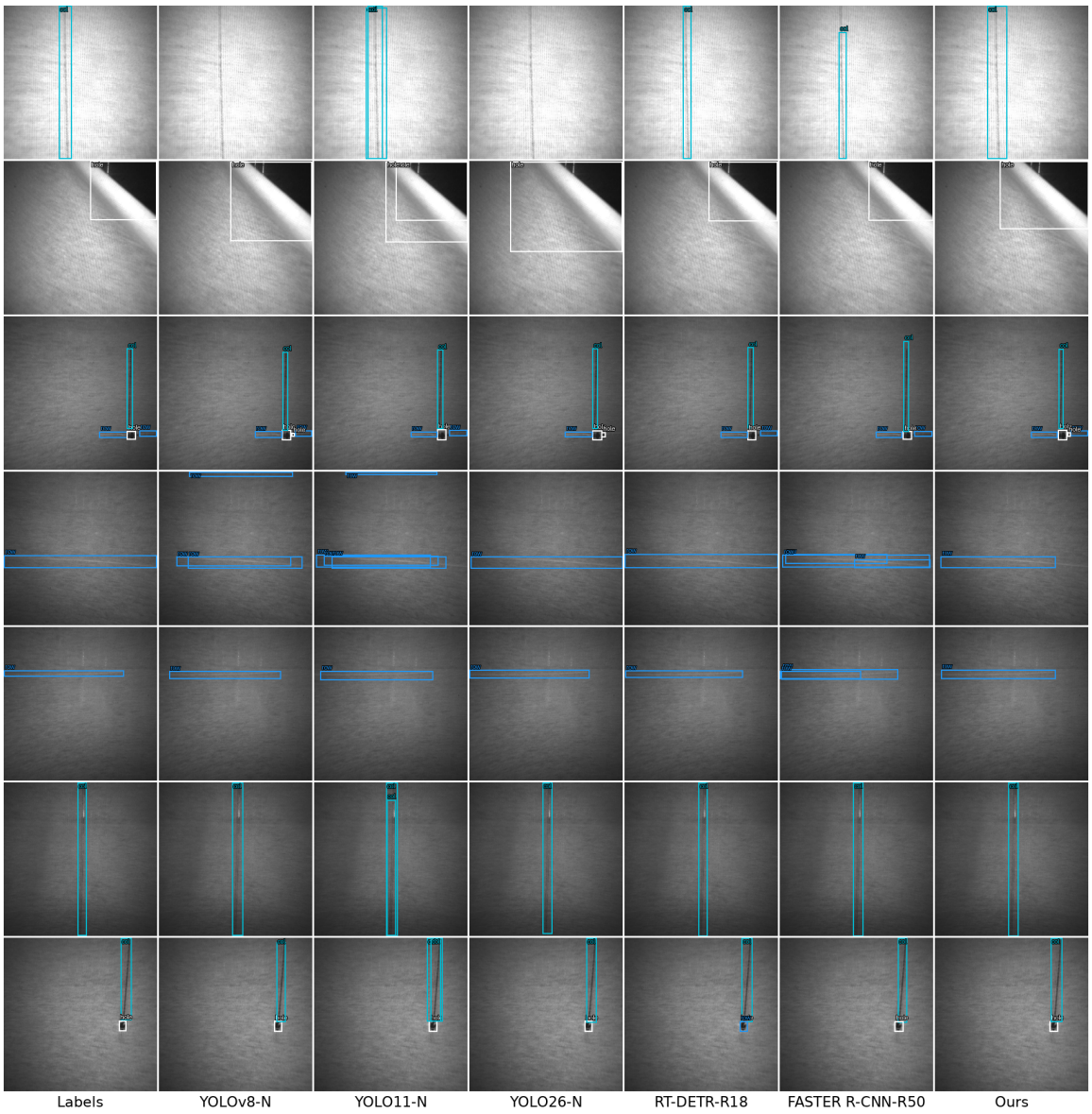

Fig. 7. Selected test predictions with a common rendering style and a confidence threshold of 0.25. Columns show ground-truth labels, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and TEFR-YOLO (Ours). The threshold is for display and does not define AP evaluation.

D. Deployment and Scope of the Evidence

The deployed TEFR-YOLO achieves 28 FPS on the RK3566 edge device, demonstrating that the modified detector can operate on the target embedded platform. This measurement complements the parameter and operation counts, because NPU execution also depends on operator support, data movement, and runtime scheduling. The system connects local predictions to wired machine control and preserves defect images for operator review and cloud analysis. The reported throughput is not a measurement of mechanical stopping time or a guarantee of complete detection coverage under every machine speed and camera arrangement.

The evaluation is bounded by the available production-line dataset. Its small, imbalanced test set limits conclusions about rare defect variations and transfer to other materials or acquisition conditions. Because the retained images contain visible defects, image-level AP does not quantify false-stop frequency during long defect-free production intervals. Likewise, the floating-point detection results do not constitute a paired assessment of quantization-induced accuracy changes. Larger independent production samples, quantized-model accuracy evaluation, and finer-grained module ablations would extend the evidence beyond the present detector comparison and deployment demonstration.

V. Conclusion

We have presented TEFR-YOLO and a production-line defect detection system for circular knitting machines. The detector combines directional texture encoding in the backbone with CARAFE-based local feature reassembly in the top-down pyramid, addressing texture representation and reconstruction within a compact YOLO architecture. On the production-line

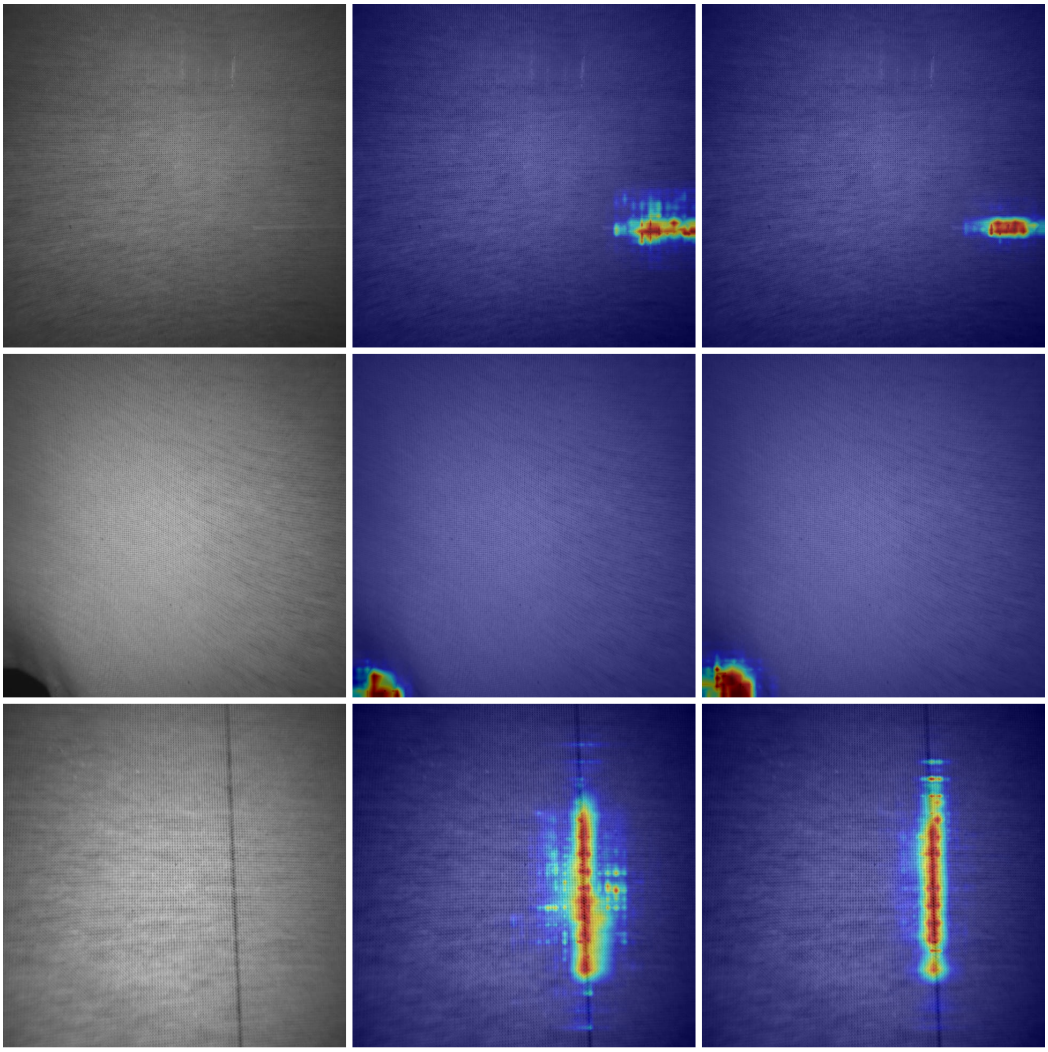

Fig. 8. Illustrative LayerCAM-based overlays. Columns: original image, YOLOv8n, and TEFR-YOLO (Ours). Rows, from top to bottom, are selected validation, training, and test examples. Both models use the same visualization procedure; these selected maps are not a quantitative test-set comparison.

test set, TEFR-YOLO raises AP from 59.3% to 66.3% and AP50 from 94.1% to 96.9%, with 3.39 M parameters and approximately 8.9 G FLOPs. Deployment on the RK3566 edge device achieves 28 FPS. The integrated system connects defect detection during machine operation with automatic local stop control, operator review, and cloud-based quality analysis. Future work will examine broader production conditions, quantized-model accuracy, and sustained operation on defect-free and defective fabric, together with more detailed analyses of the two modules.

REFERENCES

[1] Cotton Incorporated, "Knit basics," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/knit-basics/. Accessed: Oct. 8, 2026.

[2] The Woolmark Company, "Circular knit technology," [Online]. Available: https://www.woolmark.com/industry/product-development/processing-innovations/circular-knitting-technology/. Accessed: Oct. 8, 2026.

[3] Cotton Incorporated, "Single and double knits," CottonWorks, [Online]. Available: https://cottonworks.com/learning-hub/knitting/single-and-double-knits/. Accessed: Oct. 8, 2026.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.

[5] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[6] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 7464–7475.

[7] S. Gao, M. Chu, and L. Zhang, "A detection network for small defects of steel surface based on YOLOv7," *Digital Signal Processing*, vol. 149, p. 104484, 2024.

[8] C. Li, A. Xu, Q. Zhang, and Y. Cai, "Steel surface defect detection method based on improved YOLOX," *IEEE Access*, vol. 12, pp. 37 643–37 652, 2024.

[9] X. Dong, C. Zhang, J. Wang, Y. Chen, and D. Wang, "Real-time detection of surface cracking defects for large-sized stamped parts," *Computers in Industry*, vol. 159, p. 104105, 2024.

[10] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Textile Research Journal*, vol. 94, no. 1–2, pp. 225–245, 2024.

[11] H. Yang, J. Jing, Z. Wang, Y. Huang, and S. Song, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Textile Research Journal*, vol. 94, no. 1–2, pp. 49–68, 2024.

[12] X. Luo, Z. Cheng, Q. Ni, R. Tao, and Y. Shi, "Defect detection algorithm

for fabric based on deformable convolutional network," *Textile Research Journal*, vol. 93, no. 9–10, pp. 2342–2354, 2023.

[13] Z. Zhang, T. Huang, W. Zhang, Y. Yang, and Q. Yu, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, vol. 40, no. 12, pp. 5919–5944, 2025.

[14] L. Zhou, B. Ma, Y. Dong, Z. Yin, and F. Lu, "DCFE-YOLO: A novel fabric defect detection method," *PLOS ONE*, vol. 20, no. 1, p. e0314525, 2025.

[15] X. Yue, Q. Wang, L. He, Y. Li, and D. Tang, "Research on tiny target detection technology of fabric defects based on improved YOLO," *Applied Sciences*, vol. 12, no. 13, p. 6823, 2022.

[16] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, no. 6, pp. 890–898, 2024.

[17] C. Chen, Q. Zhou, S. Li, D. Luo, and G. Tan, "Fabric defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 3–4, pp. 235–251, 2025.

[18] T. Wang, N. Dai, X. Hu, K. Xu, and Y. Yuan, "SRSE-YOLO: An improved fabric defect detection method for YOLOv8," *Textile Research Journal*, 2025.

[19] Z. Gao, S. Zhang, J. Su, B. Li, J. Wang, and Z. Song, "Research on improved YOLOv8 fabric surface defect detection algorithm," *Textile Research Journal*, vol. 96, no. 17–18, 2026.

[20] S. Xu, C. Wan, W. Feng, J. Li, and J. Gao, "YOLO-DLL: An effective detection method for fabric defects," *Textile Research Journal*, 2025.

[21] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[22] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[23] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[24] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[25] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8759–8768.

[26] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[28] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.

[29] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[30] Rockchip, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[31] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[32] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

[33] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025.

[34] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[35] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[36] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[37] W. Lv, Y. Zhao, Q. Chang, K. Huang, G. Wang, and Y. Liu, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.

[38] P.-T. Jiang, C.-B. Zhang, Q. Hou, M.-M. Cheng, and Y. Wei, "LayerCAM: Exploring hierarchical class activation maps for localization," *IEEE Transactions on Image Processing*, vol. 30, pp. 5875–5888, 2021.